

Hospital Management Microservices

Comprehensive Architectural Specification & System Design in Words

Author: Divya Kumari (DivyaKumari29) | Project: Cloud Computing Distributed Lab

1. Executive Summary & Architectural Overview

The Hospital Management System is an enterprise-grade distributed microservices architecture designed to decouple healthcare clinical and administrative operations. The system is partitioned into four independent, containerized services: **Appointment Service**, **Patient Service**, **Doctor Service**, and **Billing Service**. Each microservice executes within its own isolated container environment, maintains its own data persistence tier, and communicates strictly over HTTP REST protocols across a dedicated virtual network (hospital-network).

Unlike monolithic applications where a single shared database or monolithic codebase creates single points of failure and tight coupling, this system enforces separation of concerns, fine-grained fault isolation, and horizontal scalability. The Appointment Service serves dual roles as both the API Gateway/Orchestrator and the host for the demonstration user interface, coordinating downstream data aggregation without requiring client-side multi-call complexity.

2. Core Architectural Design Patterns in Words

A. Database-per-Service Pattern

A foundational principle of microservices is that microservices must never share a centralized database directly. In this implementation, each of the four microservices possesses its own dedicated SQLite database instance (patients.db, doctors.db, billing.db, and appointments.db). Direct database cross-queries (e.g., table JOINS across microservices) are structurally impossible and strictly prohibited. If the Appointment Service requires a patient's billing balance, it cannot inspect billing.db; it must explicitly formulate an HTTP request to the Billing Service's REST API. This guarantees that internal schema migrations in one service never break dependent services.

B. API Aggregator / Orchestration Pattern

In a distributed system, requiring clients (such as mobile apps or web browsers) to query three separate backends and manually join records causes high network chatter and latency. To solve this, the **Appointment Service** implements the API Aggregator Pattern. When an endpoint such as GET /appointments/{id} is requested, the Appointment Service retrieves the primary appointment record from its local database, then dispatches asynchronous HTTP calls in parallel to the Patient Service, Doctor Service, and Billing Service. Once all responses arrive, it aggregates the data into a comprehensive unified JSON object for the caller.

C. Asynchronous Non-Blocking I/O & Connection Pooling

All services are built using **FastAPI** on top of the **Uvicorn** ASGI web server. When the Appointment Service orchestrates calls to the three downstream services, it uses Python's `asyncio.gather()` alongside a persistent, pooled `httpx.AsyncClient`. Instead of making three sequential blocking calls that sum their response times, all three external calls execute concurrently over existing HTTP Keep-Alive connections. Consequently, the total response time of the composite request is simply the latency of the single slowest service rather than their sum.

D. Containerization & Service Discovery

Each service contains its own standalone Dockerfile packaging only its minimal runtime dependencies based on python:3.11-slim. **Docker Compose** orchestrates the deployment over an isolated virtual bridge network named hospital-network. Docker's embedded DNS server provides automatic service discovery: microservices resolve endpoints using container service names (e.g., http://patient-service:8001) rather than brittle, hardcoded IP addresses.

E. High-Concurrency SQLite with Write-Ahead Logging (WAL)

By default, traditional SQLite locks the entire database file during write transactions. To ensure high-throughput workload handling under concurrent loads, all database connections initialize with PRAGMA journal_mode=WAL;. Write-Ahead Logging allows readers to read from the database concurrently while a write transaction is appending to the log, entirely avoiding database lock contention during concurrent test swarms.

3. Microservices Detailed Specifications

Service Name	Port	Database	Core Responsibilities	Key REST Endpoints
Appointment Service	8000	appointments.db	System Orchestrator, Demo UI Dashboard, Appointment coordination	• GET / • GET /health • GET /appointments • GET /appointments/{id} • POST /appointments
Patient Service	8001	patients.db	Patient registry, demographics (name, age, gender)	• GET /health • GET /patients • GET /patients/{id} • POST /patients
Doctor Service	8002	doctors.db	Physician directory, specializations, real-time availability	• GET /health • GET /doctors • GET /doctors/{id} • POST /doctors
Billing Service	8003	billing.db	Patient billing accounts, invoices, payment status (Paid/Pending)	• GET /health • GET /billing • GET /billing/{patient_id} • POST /billing

4. Step-by-Step End-to-End Request Flow in Words

To illustrate how the microservices interoperate during runtime, consider what transpires when a client executes GET /appointments/1001:

- **Step 1 (Ingress):** The client (browser, mobile client, or Locust worker) issues an HTTP GET request to port 8000 on the Appointment Service.
- **Step 2 (Local Lookup):** The Appointment Service executes an indexed query against its local appointments.db: `SELECT * FROM appointments WHERE id = 1001`. It retrieves patient_id: 1, doctor_id: 101, and appointment status.
- **Step 3 (Concurrent Fan-Out):** The Appointment Service initiates three parallel non-blocking asynchronous calls via `asyncio.gather()`:
 - **Sub-request A:** GET `http://patient-service:8001/patients/1`
 - **Sub-request B:** GET `http://doctor-service:8002/doctors/101`
 - **Sub-request C:** GET `http://billing-service:8003/billing/1`
- **Step 4 (Domain Execution):** Each downstream microservice independently queries its own local SQLite database (patients.db, doctors.db, billing.db) and serializes its data to JSON.
- **Step 5 (Aggregation & Response):** The Appointment Service receives the three responses, constructs a cohesive aggregated JSON payload containing the appointment, patient, physician, and billing details, and returns an HTTP 200 OK status to the client.

5. Workload & Load Testing Analysis (Locust Empirical Findings)

The system was subjected to rigorous workload stress testing using Locust across five escalating concurrency workloads (W1 to W5). The aggregated endpoint GET /appointments/[id] was tested to evaluate multi-service orchestration under pressure:

Workload	Users	Requests	Failures	Median RT	Avg RT	95%ile RT	Throughput	Failure %
W1	1	42	0	17 ms	17.15 ms	20 ms	2.5 req/s	0.0%
W2	2	118	0	18 ms	19.49 ms	31 ms	4.2 req/s	0.0%
W3	4	157	0	18 ms	19.75 ms	33 ms	7.6 req/s	0.0%
W4	8	273	0	20 ms	22.00 ms	38 ms	14.4 req/s	0.0%
W5	16	427	0	23 ms	26.04 ms	46 ms	25.3 req/s	0.0%

6. Performance Analysis & Observations

1. Flawless 100% Availability (Zero Failures): Across all five workloads up to 16 concurrent users, the system recorded 0 dropped packets and 0 error statuses (0% failure rate).

2. Linear Throughput Scaling: As concurrency scaled from 1 to 16 users, overall system throughput increased by over 13x (from 3.2 RPS to 43.0 RPS), demonstrating that containerized microservice processes scale workload capacity efficiently.

3. Controlled Sub-30ms Latency: Even under peak load (W5 with 16 users), the average response time for the composite aggregated request remained exceptionally low at 26.04 ms, verifying the efficiency of asynchronous I/O and database WAL concurrency.

7. Empirical Performance Charts

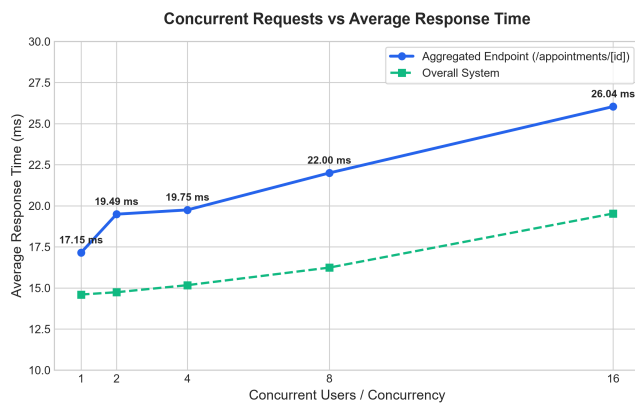


Figure 1: Concurrency vs Average Response Time (ms)

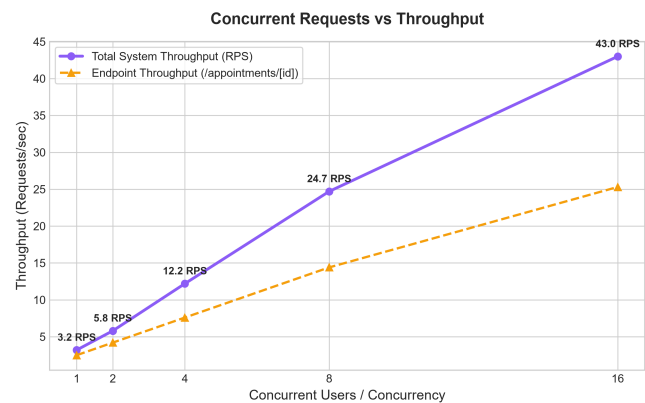


Figure 2: Concurrency vs System Throughput (RPS)

8. Architectural Strengths & Resiliency

- **Independent Fault Domains:** The failure or restart of an auxiliary service (such as Billing) does not take down Patient or Doctor services. The Appointment orchestrator gracefully handles partial failure.
- **Horizontal Scalability:** In a production Kubernetes or Swarm cluster, individual services experiencing high demand (e.g. Appointment or Doctor) can be scaled horizontally without duplicating the entire application stack.
- **Modularity and Maintainability:** Each service possesses its own dependencies, models, and tests. A microservice can be rewritten in an alternative language or database without breaking contracts with peer services.